Centro Paula Souza

Faculdade de Tecnologia de Votorantim

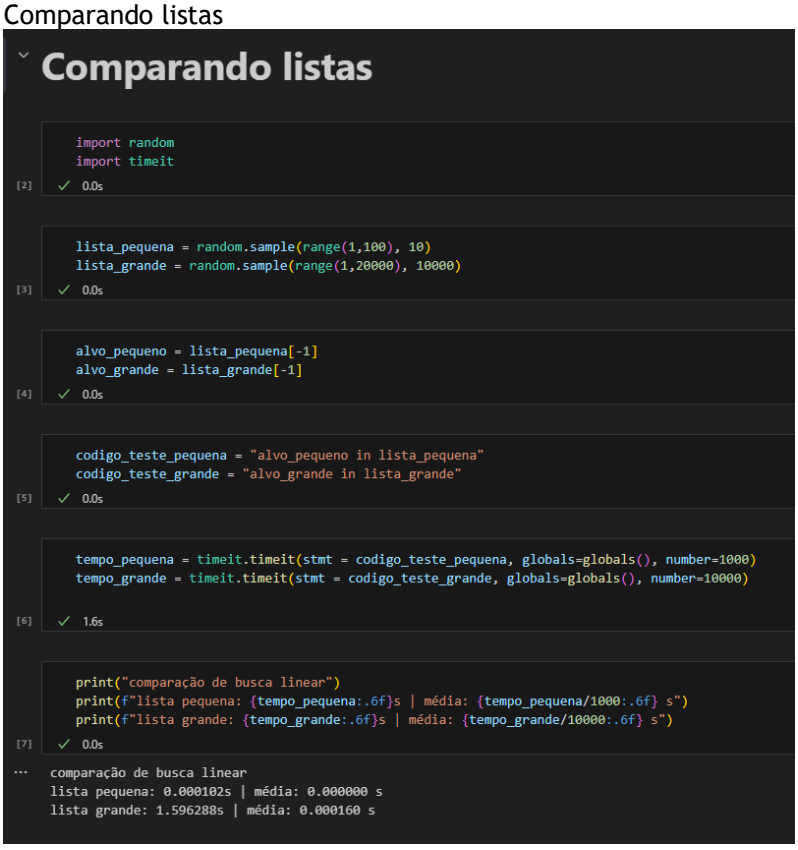
Curso de Ciência de Dados para Negócios

Aluno: Ana Elisa Rubinato Cavalheiro

Técnicas de Programação

1- Poste os exemplos criados em sala de aula:

- Comparando listas



Comparando listas

```
import random
import timeit
```
[2] ✓ 0.0s

```
lista_pequena = random.sample(range(1,100), 10)
lista_grande = random.sample(range(1,20000), 10000)
```
[3] ✓ 0.0s

```
alvo_pequeno = lista_pequena[-1]
alvo_grande = lista_grande[-1]
```
[4] ✓ 0.0s

```
codigo_teste_pequena = "alvo_pequeno in lista_pequena"
codigo_teste_grande = "alvo_grande in lista_grande"
```
[5] ✓ 0.0s

```
tempo_pequena = timeit.timeit(stmt = codigo_teste_pequena, globals=globals(), number=1000)
tempo_grande = timeit.timeit(stmt = codigo_teste_grande, globals=globals(), number=10000)
```
[6] ✓ 1.6s

```
print("comparação de busca linear")
print(f"lista pequena: {tempo_pequena:.6f}s | média: {tempo_pequena/1000:.6f} s")
print(f"lista grande: {tempo_grande:.6f}s | média: {tempo_grande/10000:.6f} s")
```
[7] ✓ 0.0s

```
comparação de busca linear
lista pequena: 0.000102s | média: 0.000000 s
lista grande: 1.596288s | média: 0.000160 s
```

- Comparando listas e conjuntos

Comparando listas e conjuntos

```python
import random
import timeit
import matplotlib.pyplot as plt

# Gerando os dados
dados = random.sample(range(1, 20000), 10000)
alvo = dados[-1]  # Pegando o último elemento da lista para garantir que está presente
lista = dados  # Criando a lista
conjunto = set(dados)  # Criando o conjunto

# Definição das quantidades de buscas
quantidades = (10, 100, 1000, 5000, 10000)

# Listas para armazenar os tempos de busca
tempos_lista = []
tempos_set = []

# Medindo o tempo de busca para cada quantidade de execuções
for n in quantidades:
    tempo_lista = timeit.timeit(stmt="alvo in lista", globals=globals(), number=n)
    tempo_set = timeit.timeit(stmt="alvo in conjunto", globals=globals(), number=n)

    tempos_lista.append(tempo_lista)
    tempos_set.append(tempo_set)

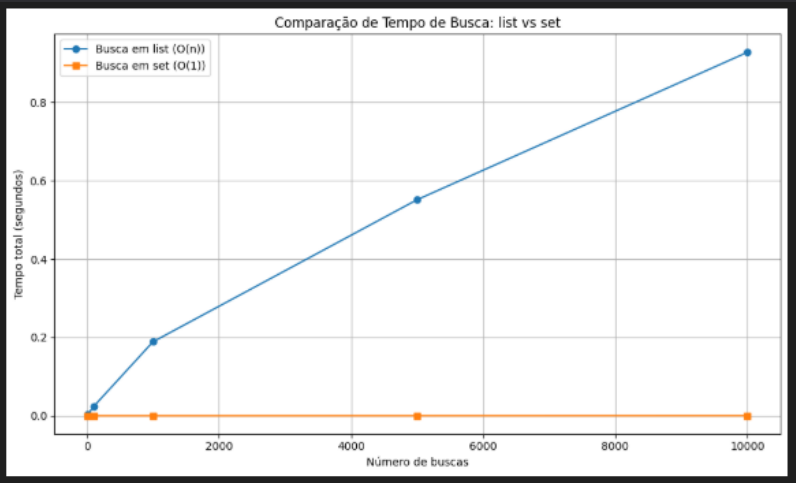
# Exibindo os resultados
print("Comparação de tempos de busca")
for i, n in enumerate(quantidades):
    print(f"{n:>5} buscas - list: {tempos_lista[i]:.6f} s | set: {tempos_set[i]:.6f} s")
```

[11] ✓ 1.7s

```
Comparação de tempos de busca
   10 buscas - list: 0.003021 s | set: 0.000003 s
  100 buscas - list: 0.023449 s | set: 0.000011 s
 1000 buscas - list: 0.188905 s | set: 0.000037 s
 5000 buscas - list: 0.551565 s | set: 0.000157 s
10000 buscas - list: 0.927805 s | set: 0.000299 s
```

```python
plt.figure(figsize=(10,6))
plt.plot(quantidades, tempos_lista, marker='o', label="Busca em list (O(n))")
plt.plot(quantidades, tempos_set, marker='s', label="Busca em set (O(1))")
plt.title("Comparação de Tempo de Busca: list vs set")
plt.xlabel("Número de buscas")
plt.ylabel("Tempo total (segundos)")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

[12] ✓ 0.2s



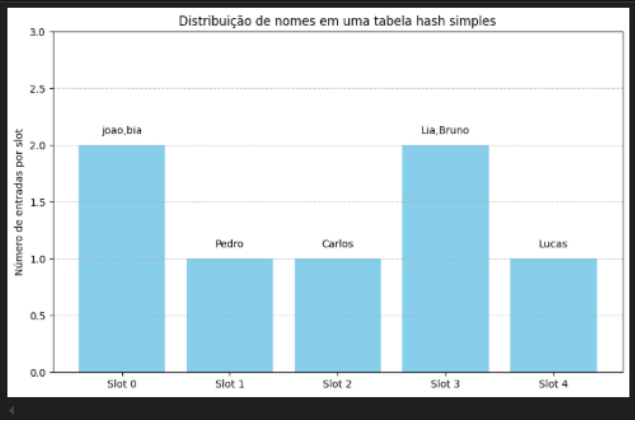
- Tabela hash

Tabela Hash

```
import matplotlib.pyplot as plt
tamanho = 5
entradas = ["joao", "bia", "Carlos", "Lucas", "Lia", "Pedro", "Bruno"]
hash_slots = [[] for _ in range(tamanho)]
def hash_simples(chave):
    return sum(ord (c) for c in chave) % tamanho
for nome in entradas:
    indice = hash_simples(nome)
    hash_slots[indice].append(nome)
```

```
labels = [f"Slot {i}" for i in range(tamanho)]
valores = [len(slot) for slot in hash_slots]
conteudo = [",".join(slot) if slot else "vazio" for slot in hash_slots]
```

```
fig, ax = plt.subplots(figsize = (10,6))
bars = ax.bar(labels, valores, color="skyblue")
for bar, texto in zip(bars, conteudo):
    yval=bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2, yval + 0.1, texto, ha='center', fontsize=10)
plt.title("Distribuição de nomes em uma tabela hash simples")
plt.ylabel("Número de entradas por slot")
plt.ylim(0, max(valores)+1)
plt.grid(axis="y", linestyle = '--', alpha = 0.7)
plt.show()
```



- Criando dicionário

Fatec
Votorantim

Centro
Paula Souza

Criando Dicionário

```
from faker import Faker
fake = Faker('pt_BR')
```

[19] ✓ 0.2s

```
clientes = {}
for _ in range(5):
  nome = fake.name()
  dados = {
      "cpf": fake.cpf(),
      "data_nascimento": fake.date_of_birth(minimum_age=18, maximum_age=(65)),
      "email": fake.email(),
      "endereco": fake.address()
  }
  clientes[nome] = dados
```

[20] ✓ 0.0s

```
for nome, info in clientes.items():
  print(f"Cliente: {nome}")
  for chave, valor in info.items():
    print(f"{chave}: {valor}")
    print("-" * 40)
```

[21] ✓ 0.0s

```
Cliente: Ana Vitória Sampaio
cpf: 081.425.369-51
----------------------------------------
data_nascimento: 1989-12-13
----------------------------------------
email: zcamara@example.org
----------------------------------------
endereco: Travessa Porto, 93
Vila Copacabana
28019168 Sá / PB
----------------------------------------
Cliente: Marcelo Moura
cpf: 295.036.478-00
----------------------------------------
data_nascimento: 1973-07-21
----------------------------------------
email: caue51@example.com
----------------------------------------
endereco: Estrada Henry Gabriel Castro, 7
Paraíso
22745329 Aparecida do Sul / AC
----------------------------------------
Cliente: Alexia Albuquerque
cpf: 210.746.839-78
----------------------------------------
...
endereco: Avenida Theo Santos, 51
Jardinópolis
67315-725 da Rosa do Sul / AM
----------------------------------------
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...